

MySQL Server & Linux Monitor Command

At MySQL Level Monitoring

1. First check currently running queries

Show full processlist;

```
SELECT
  ID,
  USER,
  HOST,
  DB,
  COMMAND,
  TIME,
  STATE,
  INFO
FROM information_schema.PROCESSLIST
WHERE COMMAND <> 'Sleep'
ORDER BY TIME DESC;
```

2. Find who is blocking whom

```
SELECT
  r.trx_id AS waiting_trx_id,
  r.trx_mysql_thread_id AS waiting_thread,
  r.trx_query AS waiting_query,
  b.trx_id AS blocking_trx_id,
  b.trx_mysql_thread_id AS blocking_thread,
  b.trx_query AS blocking_query
FROM information_schema.innodb_trx r
JOIN performance_schema.data_lock_waits w
  ON r.trx_id = w.REQUESTING_ENGINE_TRANSACTION_ID
JOIN information_schema.innodb_trx b
  ON b.trx_id = w.BLOCKING_ENGINE_TRANSACTION_ID;
```

3. Find the blocking transaction details

```
SELECT
  trx_id,
  trx_mysql_thread_id,
  trx_started,
  trx_state,
  trx_wait_started,
  trx_rows_locked,
  trx_rows_modified,
  trx_query
FROM information_schema.innodb_trx
ORDER BY trx_started;
```

4. See exactly which table/record is locked

```
SELECT
  dl.ENGINE_TRANSACTION_ID,
  dl.THREAD_ID,
```

```

dl.OBJECT_SCHEMA,
dl.OBJECT_NAME,
dl.INDEX_NAME,
dl.LOCK_TYPE,
dl.LOCK_MODE,
dl.LOCK_STATUS,
dl.LOCK_DATA
FROM performance_schema.data_locks dl;

```

5. Find deadlocks

```
SHOW ENGINE INNODB STATUS\G
```

Search for : LATEST DETECTED DEADLOCK

6. Usefull for Currently Locking & Blocking query :

```

SELECT
  r.trx_mysql_thread_id AS waiting_thread,
  r.trx_started AS waiting_started,
  r.trx_query AS waiting_query,

  b.trx_mysql_thread_id AS blocking_thread,
  b.trx_started AS blocking_started,
  b.trx_query AS blocking_query,

  TIMESTAMPDIFF(
    SECOND,
    r.trx_wait_started,
    NOW()
  ) AS wait_seconds

FROM information_schema.innodb_trx r

JOIN performance_schema.data_lock_waits w
  ON r.trx_id = w.REQUESTING_ENGINE_TRANSACTION_ID

JOIN information_schema.innodb_trx b
  ON b.trx_id = w.BLOCKING_ENGINE_TRANSACTION_ID

ORDER BY wait_seconds DESC;

```

At Linux Level Monitoring

1. Check overall server health

Uptime

This tells me the load average.

Then:

Top

CPU utilization

Memory utilization
Load average
Top CPU-consuming processes
Top memory-consuming processes

I can press P in top to sort by CPU and M to sort by memory.

2. If CPU is high

First identify which process is consuming CPU:

```
ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -20
```

Then investigate that process.

For example, if I see:

PID	USER	%CPU	COMMAND
12345	mysql	380	mysqld

I know MySQL is consuming the CPU.

I would then check the MySQL processes/queries:

```
SHOW PROCESSLIST;
```

For more detail:

```
SELECT *  
FROM information_schema.PROCESSLIST  
ORDER BY TIME DESC;
```

If necessary, I would identify the expensive query, check execution plan/indexes, and determine whether a backup, reporting query, or long-running query is responsible.

3. Check CPU per core

```
mpstat -P ALL 1 5
```

This helps me determine whether all CPUs are busy or only one CPU core is overloaded.

I'd also check:

```
iostat -xz 1
```

because high %iowait can make the server appear to have a CPU/load problem when the actual bottleneck is storage.

4. If memory is high

First:

```
free -h
```

I would primarily look at available memory and swap usage rather than only free.

Then identify the process consuming memory:

```
ps -eo pid,ppid,user,%cpu,%mem,rss,etime,cmd --sort=-%mem | head -20
```

For example:

PID	USER	%MEM	RSS	COMMAND
12345	mysql	45	14G	mysqld

Then I would investigate why MySQL is consuming that much memory — for example, buffer pool configuration, connections, large queries, temporary tables, or a memory leak.

5. Check disk space and I/O

I would also check:

`df -h`

to make sure the filesystem isn't full.

Then:

`iostat -xz 1`

to check disk performance.

If I need to find which process is doing the disk I/O:

`sudo iotop -o`

6. I would NOT immediately restart the server

This is an important interview point.

I would say:

“I wouldn't immediately reboot or kill the process. First I would collect the evidence, identify the root cause, and then take the least disruptive corrective action.”

Author : Ajit Yadav(MSSQL , MYSQL , POSTGRESQL DBA)